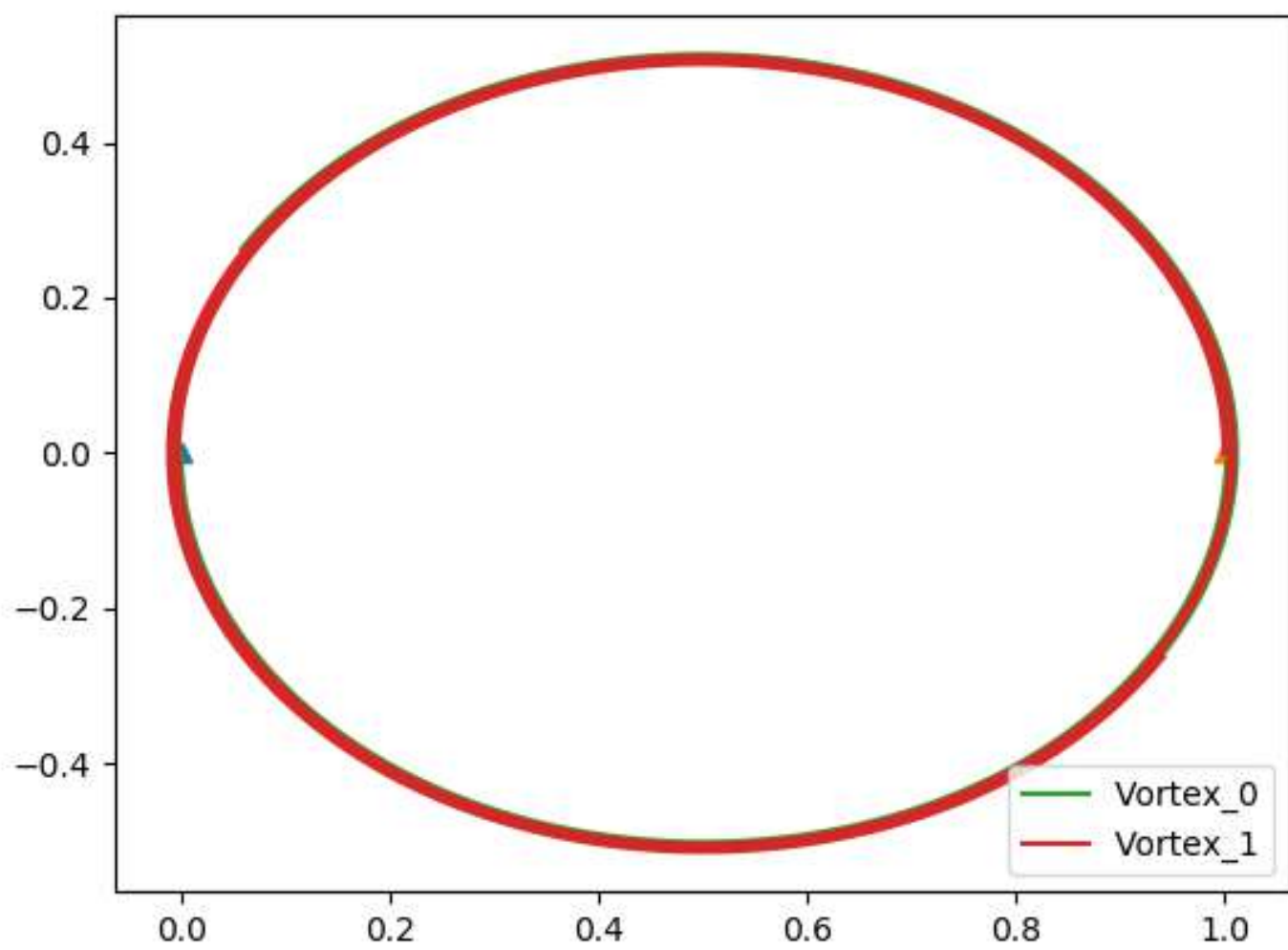
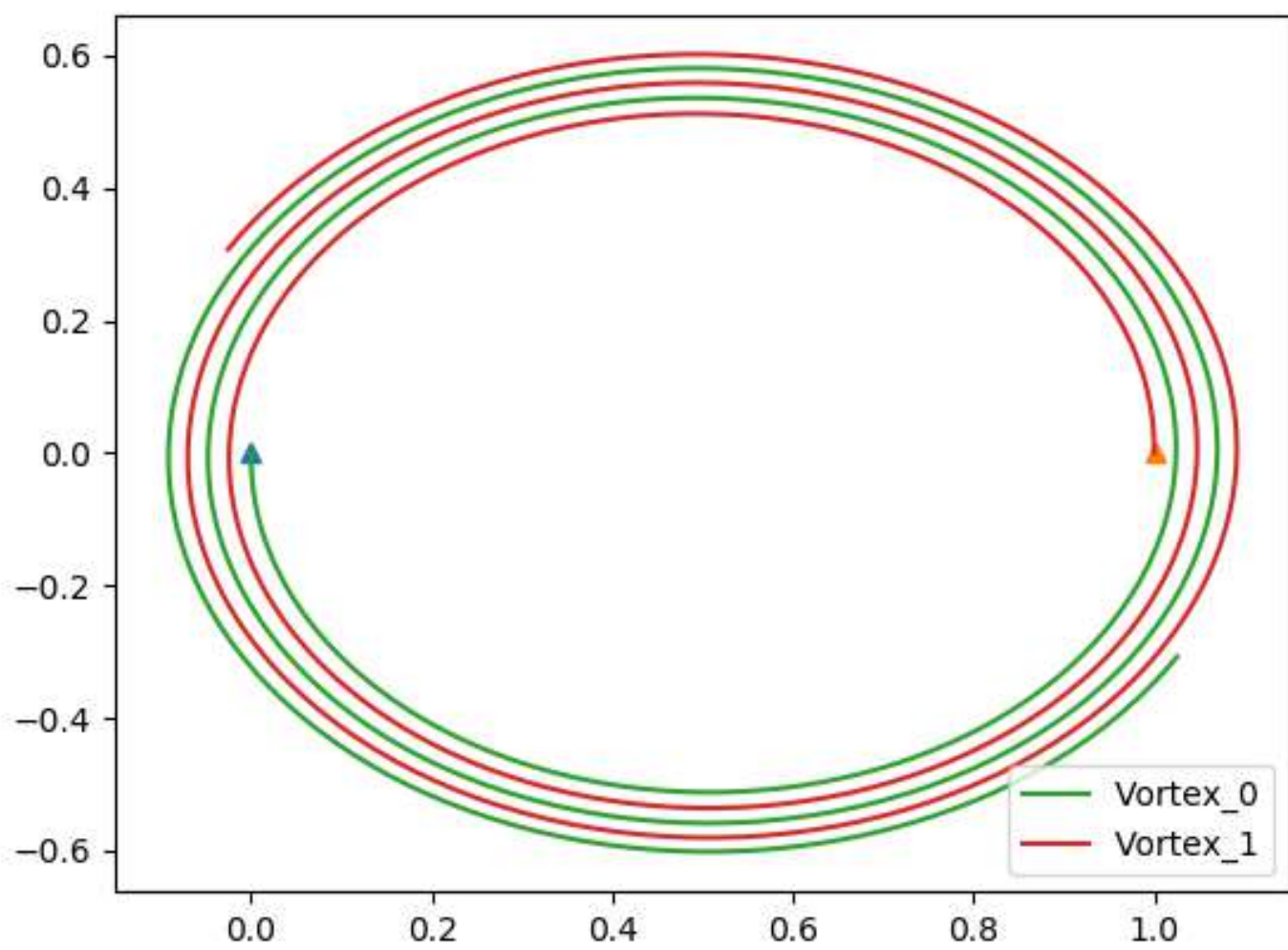
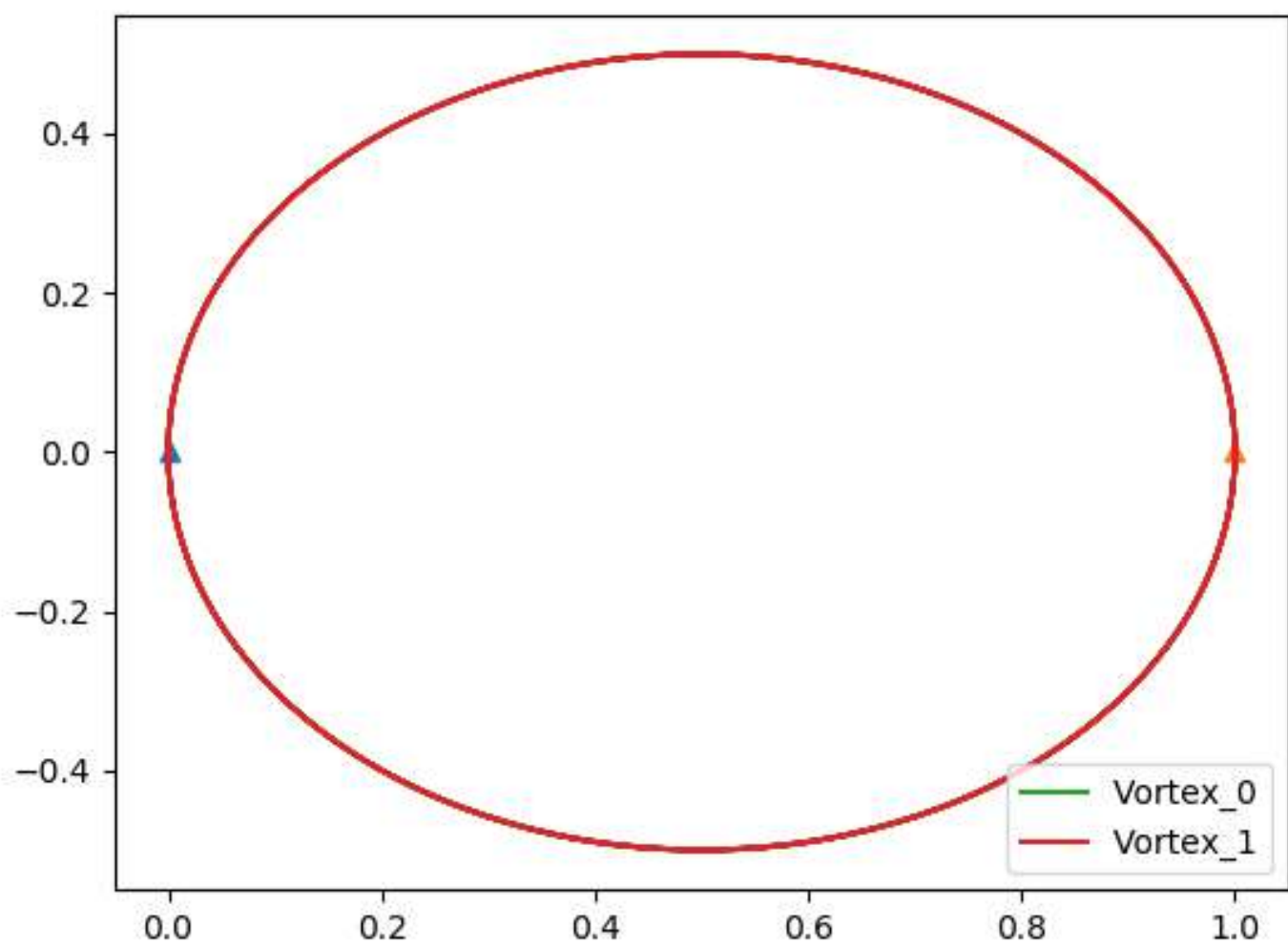
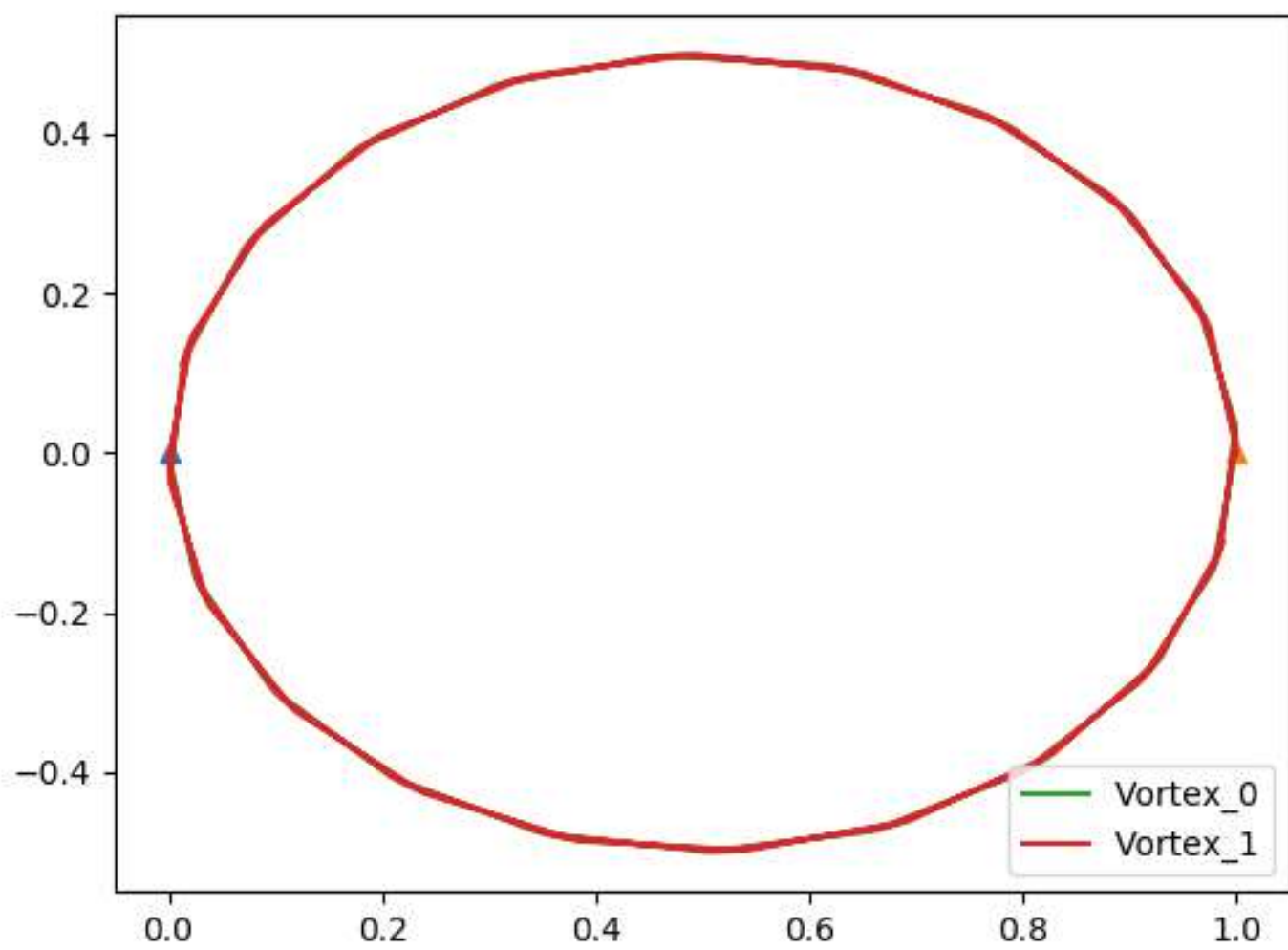
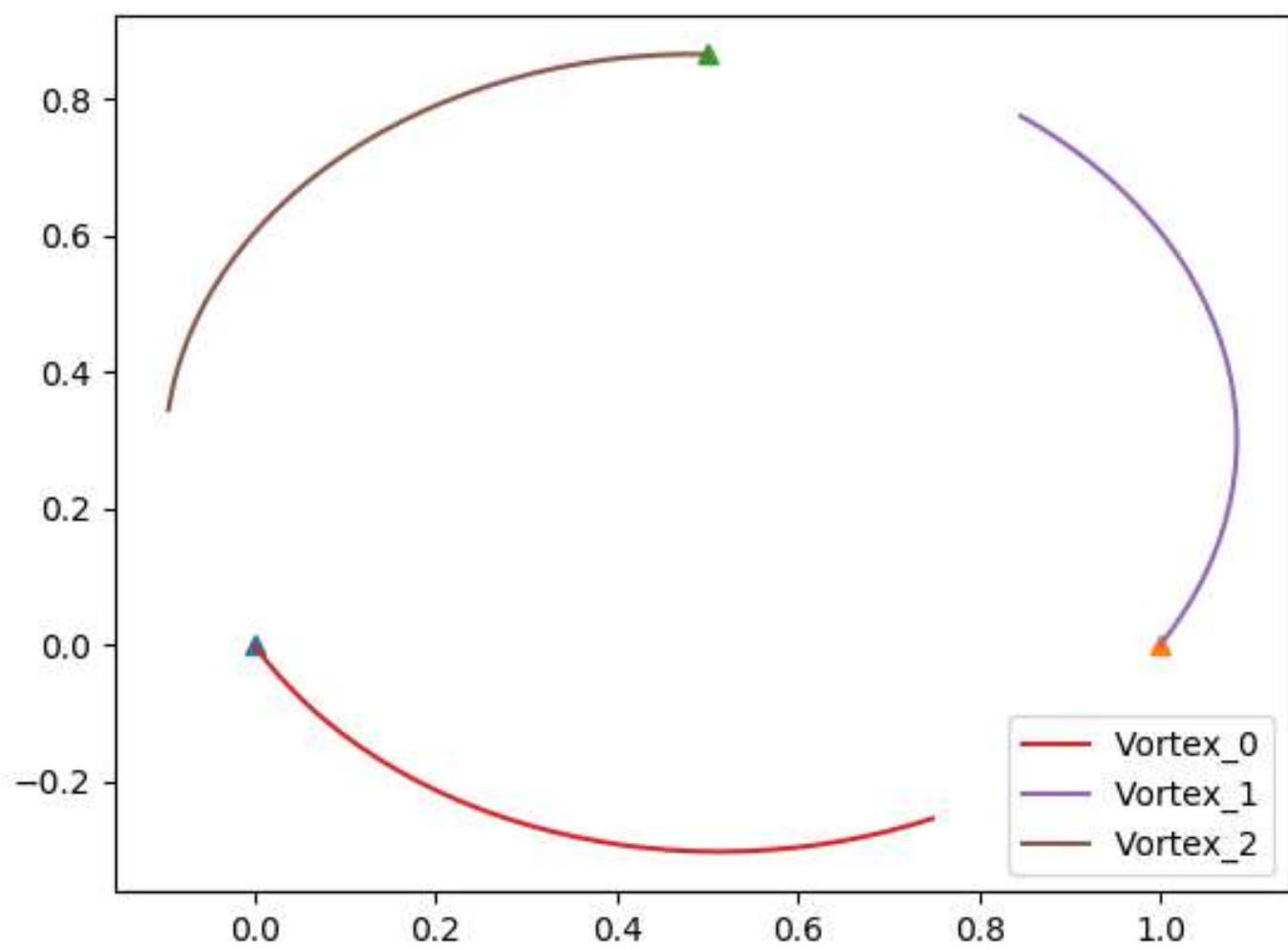


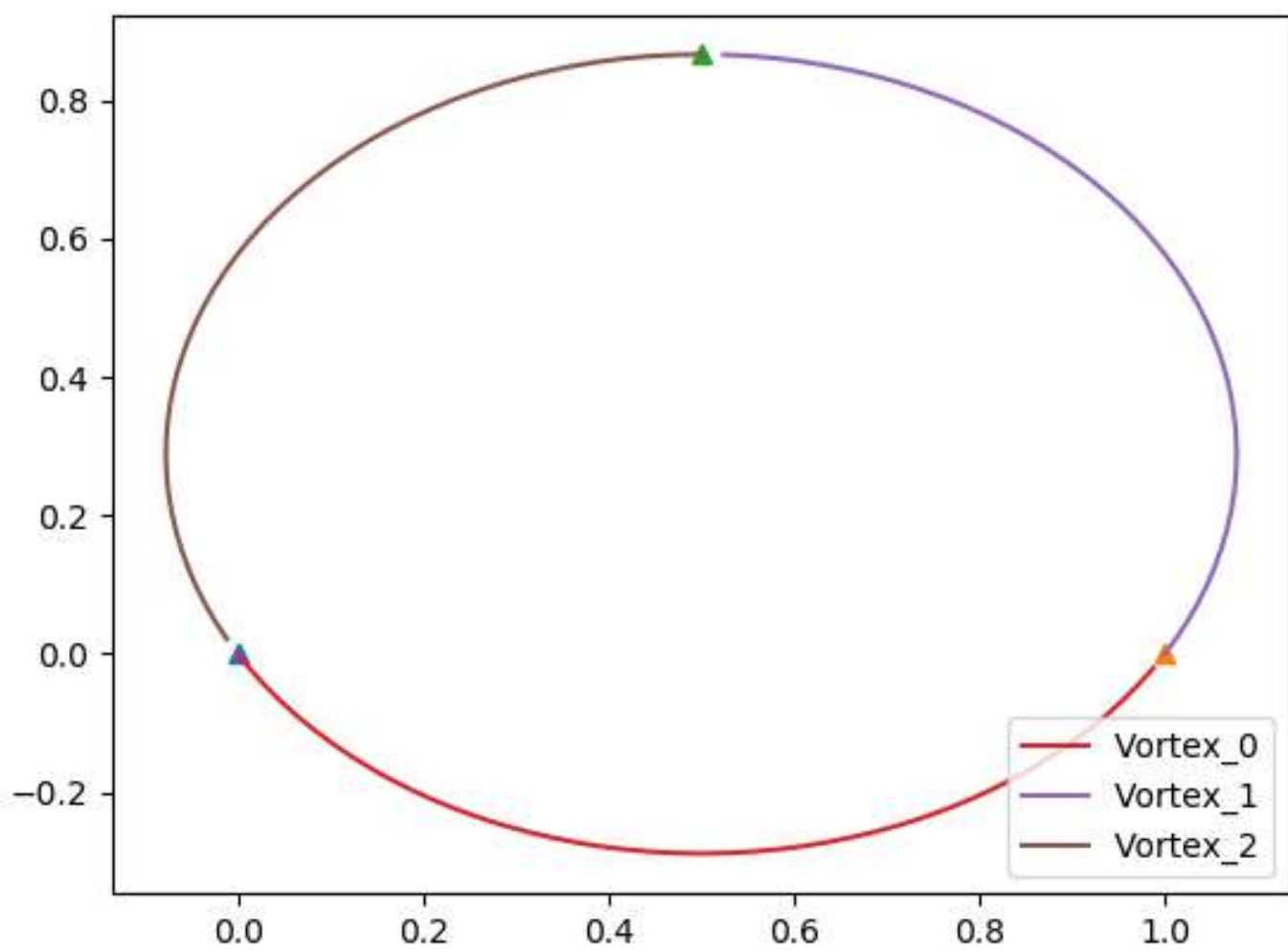


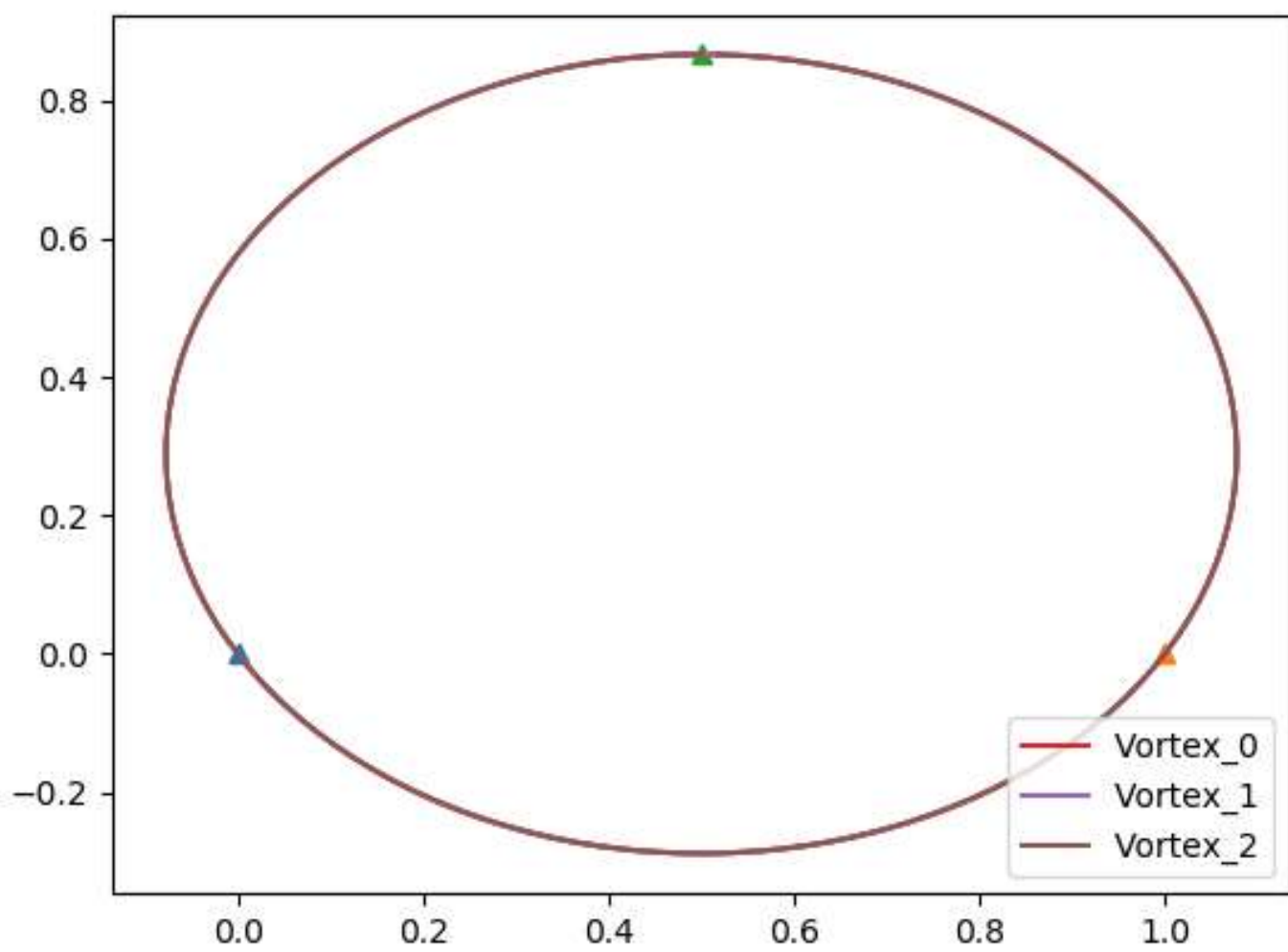


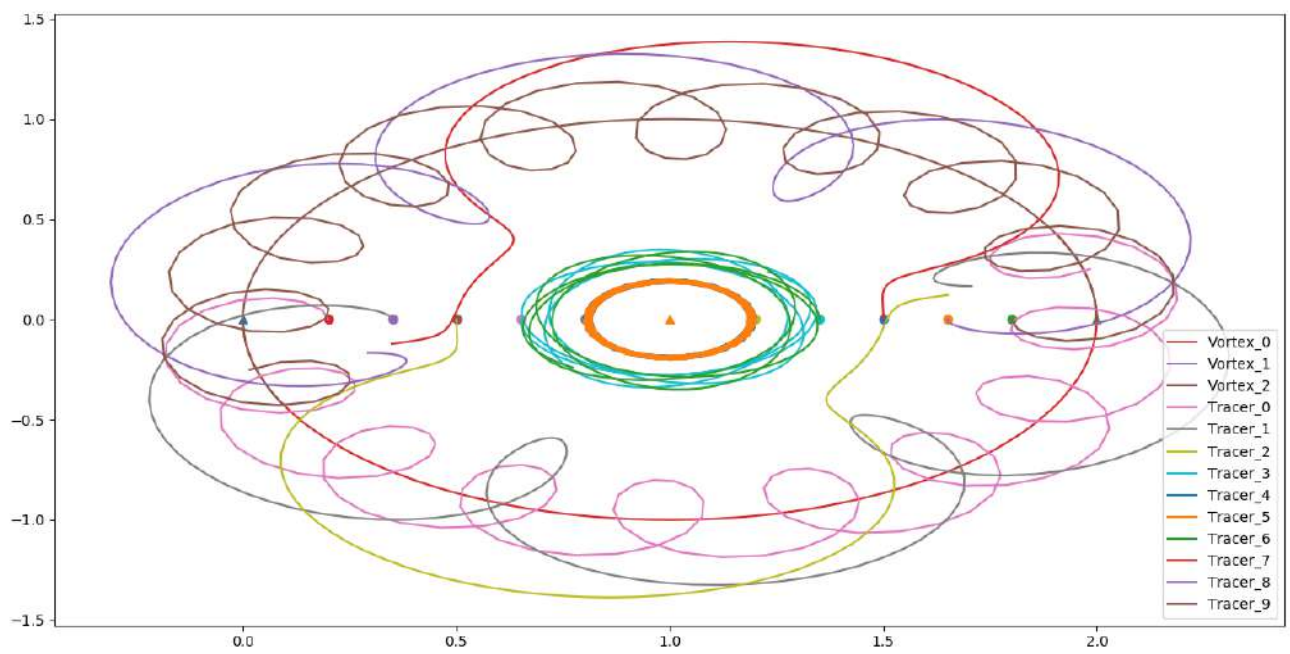


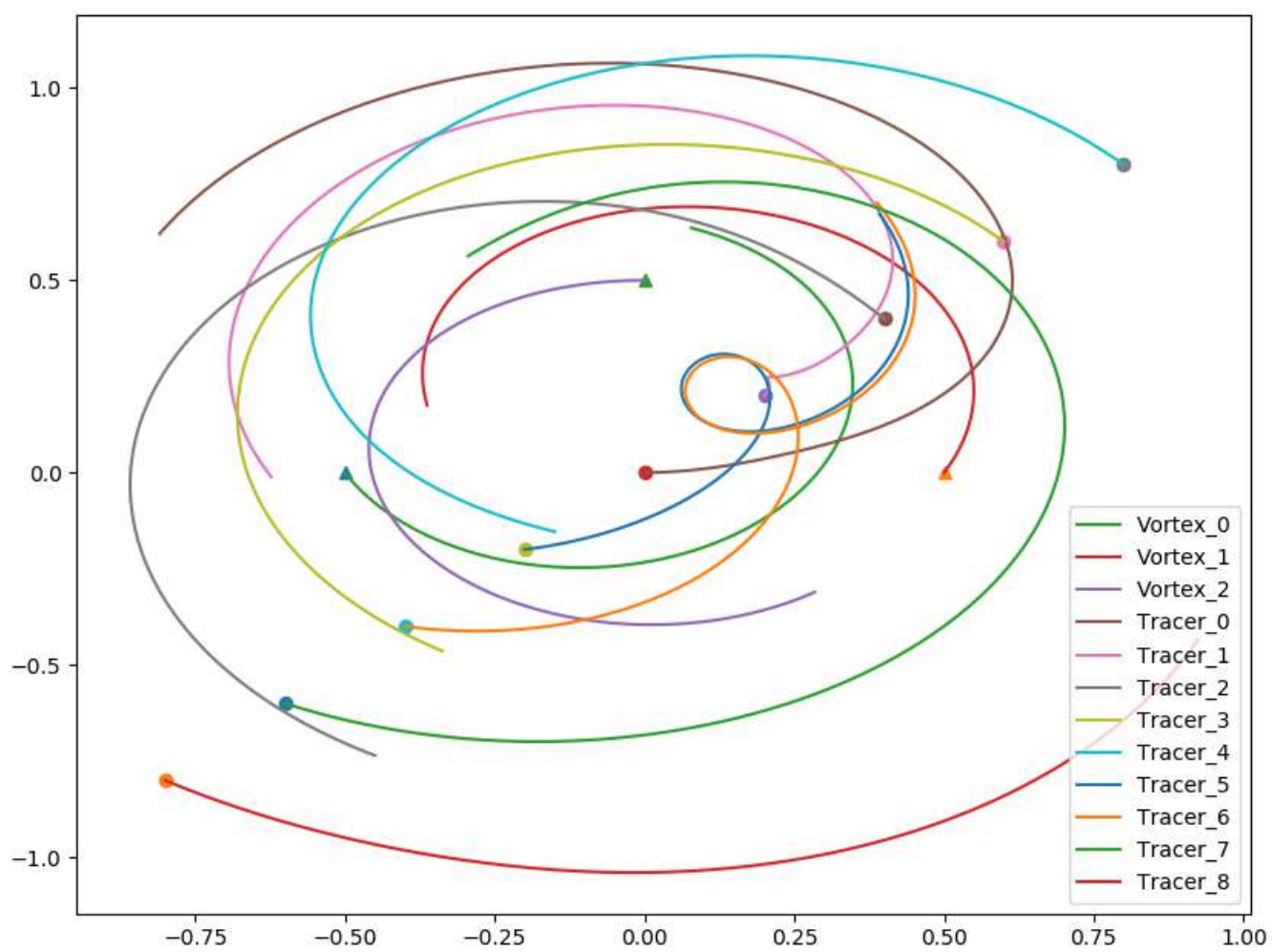


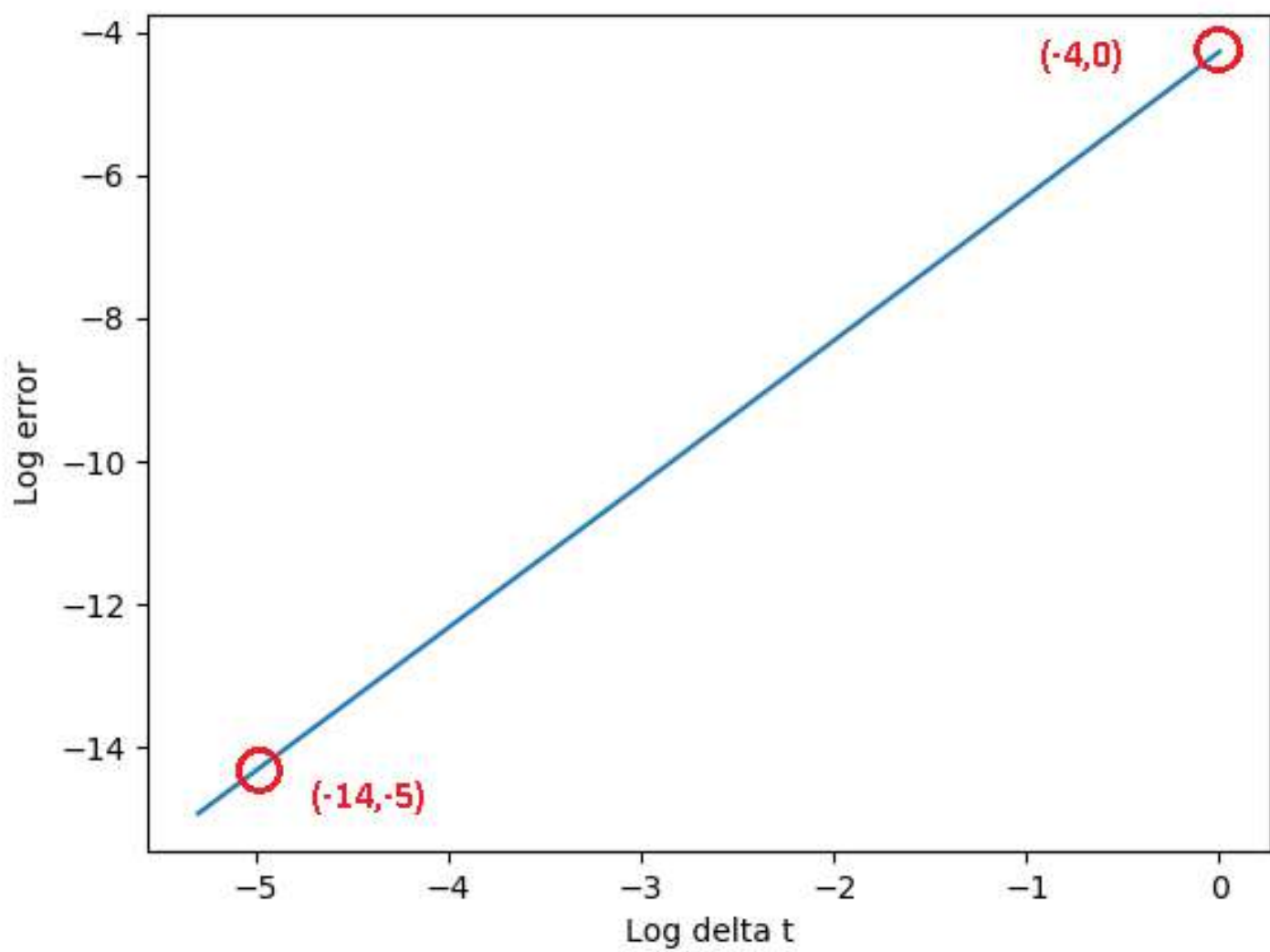


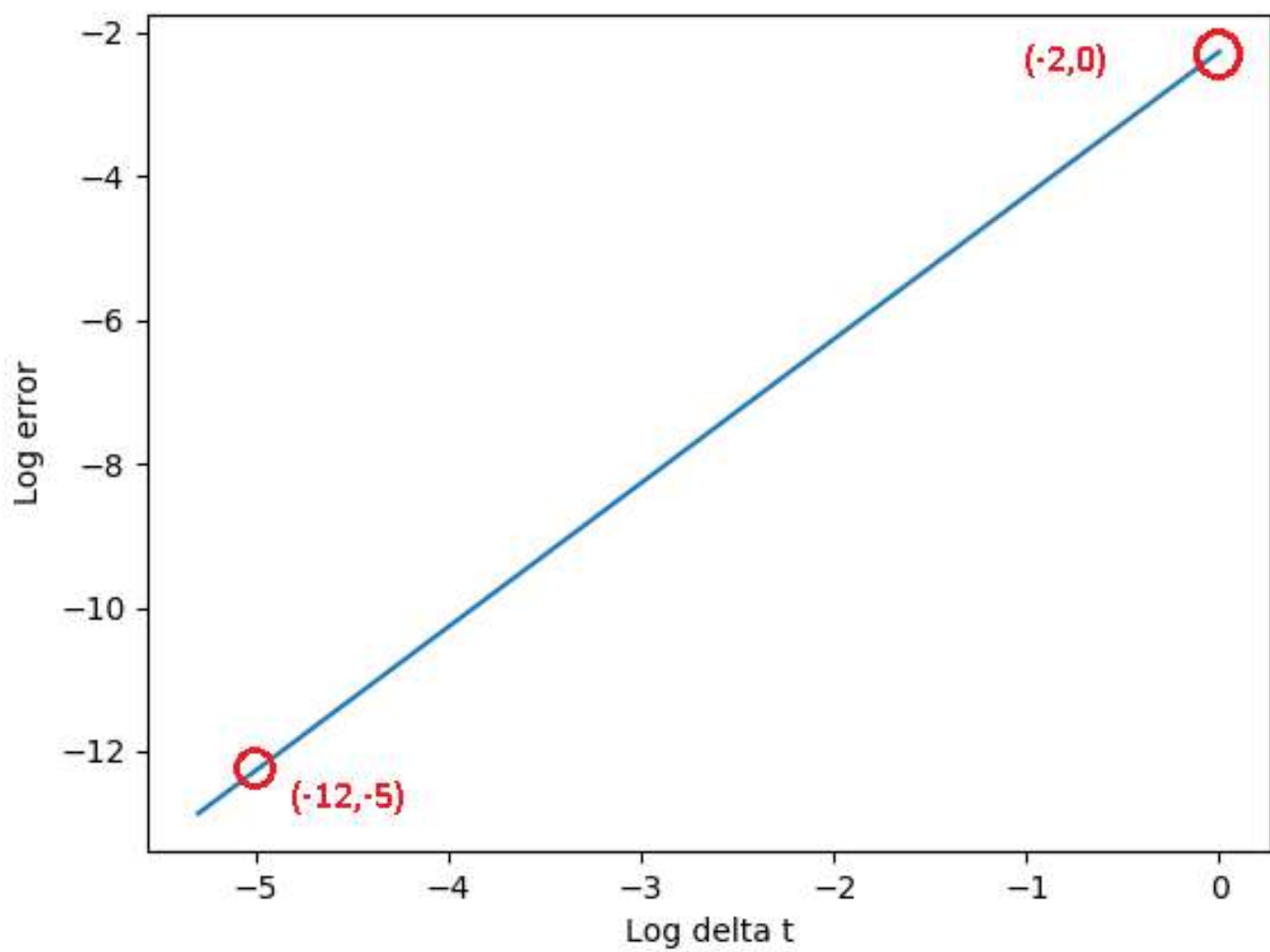












```

1  import numpy as np
2  import math as mh
3  import matplotlib.pyplot as plt
4  import cmath as cm
5
6  # Question 1#
7  """
8  No_t=0
9  N=2
10 TOS=2*(mh.pi)**2
11 cente_r=[0.5,0.0]
12 radiu_s=0.5
13 Pos_ini=[[0.0,0.0],[1.0,0.0]]
14 omega=1/mh.pi
15 """
16
17 # Question 2
18 #"""
19 No_t=0
20 N=3
21 TOS=3*13.15947252
22 #TOS=13.15947252/4
23 cente_r=[0.5,0.288675135]
24 radiu_s=0.577350269
25 Pos_ini=[[0.0,0.0],[1.0,0.0],[0.5,0.866025404]]
26 omega=0.477464829
27 #"""
28
29 # Question 3
30 """
31 No_t=10
32 N=3
33 TOS=15
34 cente_r=[1,0]
35 radiu_s=1
36 Pos_ini=[[0.0,0.0],[1.0,0.0],[2.0,0.0]]
37 Pos_tracer=[[0.2,0],[0.35,0],[0.50,0],[0.65,0],[0.8,0],[1.2,0],[1.35,0],[1.50,0],[1.65,0],
38 , [1.8,0]]
39 velocity_tracer=[0.0,0.0,0,0,0,0,0,0,0,0]
40 """
41
42 # Question 4
43 """
44 N=3
45 TOS=5
46 cente_r=[0,0]
47 radiu_s=0.5
48 Pos_ini=[[-0.5,0],[0.5,0],[0,0.5]]
49 No_t=9
50 Pos_tracer=[[0,0],[0.2,0.2],[0.4,0.4],[0.6,0.6],[0.8,0.8],[-0.2,-0.2],[-0.4,-0.4],[-0.6,-
51 0.6],[-0.8,-0.8]]
52 velocity_tracer=[0.0,0,0,0,0,0,0,0,0,0]
53 """
54
55 def Vortex_simulation(d_t):
56     #Q1
57     """
58     Pos=[[0.0,0.0],[1.0,0.0]] # Q1
59     Vel=[0.0,0.0]
60     Tau=[1.0,1]
61     """
62     #Q2
63     #"""
64     Pos=[[0.0,0.0],[1.0,0.0],[0.5,0.866025404]] # Q2
65     Vel=[0.0,0.0,0.0]
66     Tau=[1.0,1.0,1.0]
67     #"""

```

```

66 #Q3
67 """
68 Pos=[[0.0,0],[1,0],[2,0]] # Q3
69 Vel=[0.0,0,0]
70 Tau=[1.0,1,1]
71 Vel_t=[0.0,0,0,0,0,0,0,0,0,0]
72
73 Pos_t=[[0.2,0],[0.35,0],[0.50,0],[0.65,0],[0.8,0],[1.2,0],[1.35,0],[1.50,0],[1.65,0],
74 [1.8,0]]
75 """
76
77 # Q4
78 """
79 Pos=[[-0.5,0],[0.5,0],[0,0.5]]
80 Vel=[0.0,0,0]
81 Tau=[1,1,1]
82 Vel_t=[0,0,0,0,0,0,0,0,0,0]
83
84 Pos_t=[[0,0],[0.2,0.2],[0.4,0.4],[0.6,0.6],[0.8,0.8],[-0.2,-0.2],[-0.4,-0.4],[-0.6,-0.6],
85 [-0.8,-0.8]]
86 """
87
88 def velocity(i):
89     z_i=complex(Pos[i][0],Pos[i][1])
90     vel_sum=0
91     for j in range(N):
92         if j!=i:
93             z_j=complex(Pos[j][0],Pos[j][1])
94             vel_sum+=(complex(0,-0.5/(mh.pi)))*Tau[j]/(z_i-z_j)
95
96     Vel_i=vel_sum.conjugate()
97     return Vel_i
98
99 def velocity_t(i):
100     z_i=complex(Pos_t[i][0],Pos_t[i][1])
101     vel_sum=0
102     for j in range(N):
103         z_j=complex(Pos[j][0],Pos[j][1])
104         vel_sum+=(complex(0,-0.5/(mh.pi)))*Tau[j]/(z_i-z_j)
105
106     Vel_i=vel_sum.conjugate()
107     return Vel_i
108
109 Path_tracker=[]
110 Path_tracker_t=[]
111 tim=0
112 if No_t!=0:
113     # Euler method
114     """
115     while tim<TOS:
116         Path_tracker=np.append(Path_tracker,[Pos])
117         Path_tracker_t=np.append(Path_tracker_t,[Pos_t])
118         for nop in range(No_t):
119             Vel_t[nop]=velocity_t(nop)
120         for nop in range(No_t):
121             Pos_t[nop][0]+=(Vel_t[nop]*d_t).real
122             Pos_t[nop][1]+=(Vel_t[nop]*d_t).imag
123         for nop in range(N):
124             Vel[nop]=velocity(nop)
125         for nop in range(N):
126             Pos[nop][0]+=(Vel[nop]*d_t).real
127             Pos[nop][1]+=(Vel[nop]*d_t).imag
128         tim+=d_t
129     return Path_tracker,Path_tracker_t
130     """
131
132     # RK_2
133     #"""
134     while tim<TOS+d_t:

```

```

129     Path_tracker=np.append(Path_tracker,[Pos])
130     Path_tracker_t=np.append(Path_tracker_t,[Pos_t])
131
132     # y(t)
133     for nop in range(No_t):
134         Vel_t[nop]=velocity_t(nop)
135     for nop in range(N):
136         Vel[nop]=velocity(nop)
137
138     Pos_temp=np.copy(Pos)
139     Vel_temp=np.copy(Vel)
140     Pos_t_temp=np.copy(Pos_t)
141     Vel_t_temp=np.copy(Vel_t)
142
143     # y tilda
144     for nop in range(No_t):
145         Pos_t[nop][0]+=0.5*(Vel_t[nop]*d_t).real
146         Pos_t[nop][1]+=0.5*(Vel_t[nop]*d_t).imag
147
148     for nop in range(N):
149         Pos[nop][0]+=0.5*(Vel[nop]*d_t).real
150         Pos[nop][1]+=0.5*(Vel[nop]*d_t).imag
151
152     for nop in range(No_t):
153         Vel_t[nop]=velocity_t(nop)
154
155     for nop in range(N):
156         Vel[nop]=velocity(nop)
157
158     # Y(t+1)
159     Pos_t=np.copy(Pos_t_temp)
160     Pos=np.copy(Pos_temp)
161
162     for nop in range(No_t):
163         Pos_t[nop][0]+=d_t*(Vel_t[nop]).real
164         Pos_t[nop][1]+=d_t*(Vel_t[nop]).imag
165
166     for nop in range(N):
167         Pos[nop][0]+=d_t*(Vel[nop]).real
168         Pos[nop][1]+=d_t*(Vel[nop]).imag
169
170     tim+=d_t
171     del Pos_temp
172     del Pos_t_temp
173     del Vel_temp
174     del Vel_t_temp
175     return Path_tracker,Path_tracker_t
176
177     #"""
178 if No_t==0:
179     """
180     #Euler
181     error_val=0
182     k_iter=0
183
184     while tim<TOS:
185         Path_tracker=np.append(Path_tracker,[Pos])
186         for nop in range(N):
187             Vel[nop]=velocity(nop)
188         for nop in range(N):
189             Pos[nop][0]+=(Vel[nop]*d_t).real
190             Pos[nop][1]+=(Vel[nop]*d_t).imag
191         tim+=d_t
192
193         x1 = 0.5*(1-mh.cos(omega*tim))
194         y1 = -0.5*(mh.sin(omega*tim))
195         k_iter += 1

```

```

196         error_val += ((Pos[0][0]-x1)**2 + (Pos[0][1]-y1)**2)**0.5
197     error_val = error_val/k_iter
198     return Path_tracker#,error_val
199     """
200
201     # RK_2
202     #"""
203     error_val=0
204     k_iter=0
205     while tim<TOS:
206         Path_tracker=np.append(Path_tracker,[Pos])
207
208         # y(t)
209         for nop in range(N):
210             Vel[nop]=velocity(nop)
211
212         Pos_temp=np.copy(Pos)
213         Vel_temp=np.copy(Vel)
214
215         # y tilda
216         for nop in range(N):
217             Pos[nop][0]+=0.5*(Vel[nop]*d_t).real
218             Pos[nop][1]+=0.5*(Vel[nop]*d_t).imag
219         for nop in range(N):
220             Vel[nop]=velocity(nop)
221         # Y(t+1)
222         Pos=np.copy(Pos_temp)
223         for nop in range(N):
224             Pos[nop][0]+=d_t*(Vel[nop]).real
225             Pos[nop][1]+=d_t*(Vel[nop]).imag
226
227         tim+=d_t
228
229         #x1 = 0.5*(1-mh.cos(omega*tim))
230         #y1 = -0.5*(mh.sin(omega*tim))
231         y1 = Pos_ini[2][1]-radiu_s*(1-mh.cos(omega*tim))
232         x1 = 0.5-radiu_s*mh.sin(omega*tim)
233         k_iter += 1
234         error_val += ((Pos[2][0]-x1)**2 + (Pos[2][1]-y1)**2)**0.5
235         del Pos_temp
236         del Vel_temp
237
238     error_val = error_val/k_iter
239     return Path_tracker#,error_val
240     """
241
242 def Path_plotting(d_t):
243     if No_t==0:
244         Path_track=Vortex_simulation(d_t)
245         #print(len(Path_track))
246         ## Path_line = [[path1(x),path1(y)]......]
247         Path_line=[None for iter_1 in range(N)]
248         for iter_1 in range(N):
249             Path_line[iter_1]=[[],[]]
250             for i in range(int(len(Path_track)/(2*N))):
251                 Path_line[iter_1][0].append(Path_track[2*N*i+2*iter_1])
252                 Path_line[iter_1][1].append(Path_track[2*N*i+2*iter_1+1])
253     plt.figure(1)
254     for iter_3 in Pos_ini:
255         plt.plot(iter_3[0],iter_3[1],'^')
256     for iter_1 in range(N):
257         plt.plot(Path_line[iter_1][0],Path_line[iter_1][1],'-.',label='Vortex_'+str(
258             iter_1))
259         #plt.show()
260     plt.legend(loc='lower right')
261     plt.show()

```

```

262 if No_t!=0:
263     Path_track,Path_track_t=Vortex_simulation(d_t)
264     ## Path_line = [[path1(x),path1(y)].....]
265     Path_line=[None for iter_1 in range(N)]
266     Path_line_t=[None for iter_1 in range(No_t)]
267     for iter_1 in range(N):
268         Path_line[iter_1]=[],[]
269         for i in range(int(len(Path_track)/(2*N))):
270             Path_line[iter_1][0].append(Path_track[2*N*i+2*iter_1])
271             Path_line[iter_1][1].append(Path_track[2*N*i+2*iter_1+1])
272     for iter_1 in range(No_t):
273         Path_line_t[iter_1]=[],[]
274         for i in range(int(len(Path_track_t)/(2*No_t))):
275             Path_line_t[iter_1][0].append(Path_track_t[2*No_t*i+2*iter_1])
276             Path_line_t[iter_1][1].append(Path_track_t[2*No_t*i+2*iter_1+1])
277
278     figure_1=plt.figure(1)
279     for iter_3 in Pos_ini:
280         plt.plot(iter_3[0],iter_3[1],'^')
281     for iter_3 in Pos_tracer:
282         plt.plot(iter_3[0],iter_3[1],'o')
283     for iter_1 in range(N):
284         plt.plot(Path_line[iter_1][0],Path_line[iter_1][1],label='Vortex_'+str(iter_1))
285     for iter_1 in range(No_t):
286         plt.plot(Path_line_t[iter_1][0],Path_line_t[iter_1][1],label='Tracer_'+str(iter_1))
287     plt.legend(loc='lower right')
288     plt.show()
289
290 def calculating_error(d_t):
291     Path_track,error_val=Vortex_simulation(d_t)
292     return error_val
293
294
295 def order_verification():
296     choice_dt=[1,0.5,0.1,0.05,0.01, 5e-3 ]
297     Error_val=[]
298     for i in choice_dt:
299         Error_val.append(calculating_error(i))
300     plt.figure(2)
301     plt.plot(np.log(choice_dt),np.log(Error_val))
302     plt.xlabel('Log delta t')
303     plt.ylabel('Log error' )
304     plt.show()
305
306

```